Assignment 06 - Visitor Pattern File System

Java 17, Maven, JUnit 5. Author: **Dayglovv**. GitHub: **https://github.com/Dayglovv/SDP6**

1. Introduction

The project models a file system (files and nested directories) and performs three independent operations on it using the Visitor pattern: **TotalSizeVisitor** (total size of all files), **SearchVisitor** (files matching a name criterion and a minimum size, returned as sorted relative paths) and **StatisticsVisitor** (number of files, directories and maximum depth). The structure never changes when a new operation is added.

2. Design choices

- **Double Dispatch.** `element.accept(visitor)` selects the element type; `visitor.visit(this)` then selects the overload `visit(FileNode)` or `visit(DirectoryNode)`.
- **Traversal: variant B (visitors own it).** The recursion is written once in the abstract class `TraversingVisitor`: a directory calls `child.accept(this)` for every child. This needs no `instanceof`, `getClass()`, reflection or casts, and avoids a separate walker that would have to know the element types. Concrete visitors override only small hooks (`onStart, onFile, onEnterDirectory, onLeaveDirectory`). Each top-level traversal calls `onStart()`, so visitors are reusable.
- **Tree integrity.** `DirectoryNode.add()` rejects null, duplicate names, elements that already have a parent (`IllegalStateException`), and cycles (adding a directory to itself or to its own descendant) via a walk up the parent chain. Negative sizes and bad names are rejected in constructors. `getChildren()` returns an unmodifiable copy. `attachTo()` only succeeds if the directory really contains the element, so it cannot bypass `add()`.
- **Search.** A file matches if its name contains the criterion (case-insensitive) AND `size >= minSize` (inclusive). The current path is kept in a deque (push on enter, pop on leave); the root name is not part of the path; separator is "/"; results are sorted.
- **Depth.** Depth is measured over directories: root = 0, its sub-directories = 1. This matches the assignment example (3 files, 3 directories, depth 2). The root counts as a directory; empty directories are counted.

3. UML

The class diagram is on the last page (it is not counted in the 3-page limit).

4. Clean Code practices

- **Meaningful naming:** `TotalSizeVisitor, getResults(), isSelfOrAncestor(), hasChildNamed()` say what they do; no abbreviations.
- **Single Responsibility:** nodes only hold data and guard integrity; each visitor implements exactly one operation; traversal lives only in `TraversingVisitor`.
- **Small methods:** most methods are 3-10 lines; validation is split into helpers (`validateName, matches, hasChildNamed`).
- **Encapsulation:** fields are private/final, the children list is never exposed (unmodifiable copy), the parent link is set only through a checked method, results of visitors are exposed via getters returning immutable data.
- **No type-checking chains:** there is no `instanceof`, cast or `getClass()` anywhere; polymorphism and Double Dispatch choose the behaviour.

5. Sample output (`mvn compile exec:java`)

Demo tree: Root{document.txt 1000, Pictures{photo.jpg 2000, avatar.png 500}, Documents{report.pdf 3000}, Projects{report-final.pdf 1500, report-draft.pdf 400, Empty{}}}. Output is deterministic:

```
=== TotalSizeVisitor ===
Total size: 8400 bytes

=== SearchVisitor (name contains 'report', size >= 1000) ===
Documents/report.pdf
Projects/report-final.pdf

=== StatisticsVisitor ===
Files: 6
Directories: 5
Maximum depth: 2
```

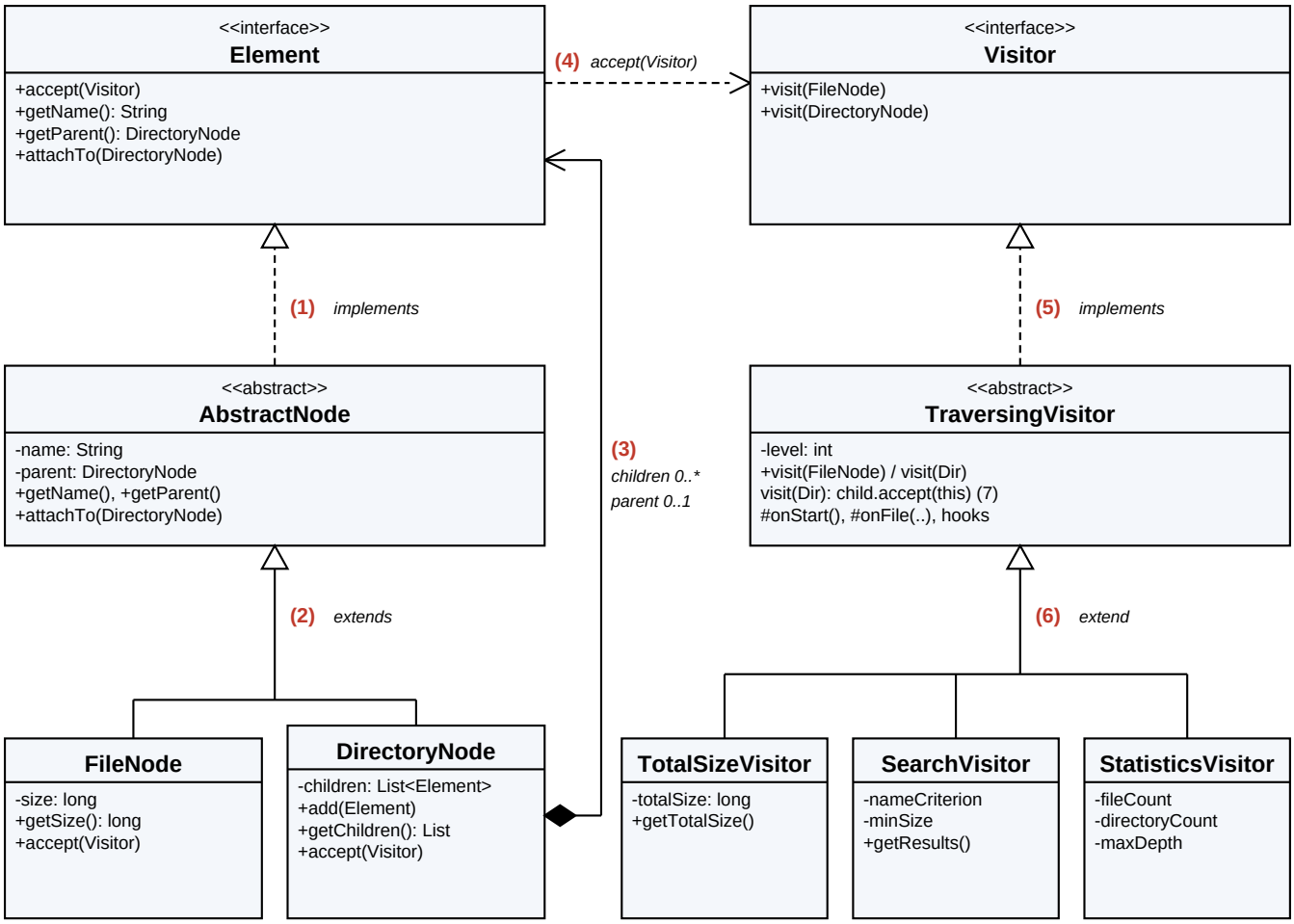
6. Tests (`mvn test`)

Classes: **VisitorTest** (12 tests) and **FileSystemTest** (11 tests): 23 tests in total, 0 failures. Coverage of the required areas: T1 correct visit() overload; T2 total size on several levels (6500 bytes); T3 both search filters, inclusive threshold; T4 relative, "/"-separated, sorted paths; T5 statistics and depth; T6 empty directories and empty search results; T7 repeated runs do not accumulate; T8 negative size, duplicate names, two parents, self-containment, cycles, unmodifiable children.

7. Trade-offs

- **Traversal inside visitors:** the loop is written once in the base class and visitors stay simple, but every visitor is forced to traverse the whole tree (an operation that wants to skip a subtree needs another hook). A separate walker would give more control but would need type knowledge or extra element methods.
- **Extensibility:** adding a new operation is easy - one new visitor class, no change to elements. Adding a new element type (e.g. SymbolicLink) is costly: a new `visit(...)` in `Visitor` and changes in all existing visitors.
- **Public attachTo():** because the interface-only API cannot hide a method, it is public but validated; the price is a slightly wider public interface.

UML class diagram



Numbers (1)-(7) in red are explained in the table below. Legend: solid + hollow triangle = inheritance; dashed + hollow triangle = implements; dashed + open arrow = dependency; solid + diamond = composition.

#	Relation	Meaning
1	AbstractNode implements Element	Common base for nodes: stores the validated name and the parent link, so FileNode and DirectoryNode do not duplicate it.
2	FileNode, DirectoryNode extend AbstractNode	Both are Elements. FileNode adds size; DirectoryNode adds the list of children and the integrity checks in add().
3	DirectoryNode owns 0..* Element (composition); Element has parent 0..1	A directory contains files and directories (children). Each element belongs to at most one directory, so the tree has no sharing and no cycles.
4	Element depends on Visitor: accept(Visitor)	Double Dispatch step 1: the element receives a visitor. Step 2: accept() calls visitor.visit(this), so Visitor.visit(FileNode) or visit(DirectoryNode) is chosen by the element type.
5	TraversingVisitor implements Visitor	Implements both visit() methods once and runs the recursion over the children, so the traversal is written in one place.
6	TotalSizeVisitor, SearchVisitor, StatisticsVisitor extend TraversingVisitor	Each concrete operation only fills in small hooks (onStart, onFile, onEnterDirectory, onLeaveDirectory); the tree classes are not changed.
7	TraversingVisitor uses DirectoryNode.getChildren()	visit(DirectoryNode) calls child.accept(this) for every child; Double Dispatch picks the right visit() without instanceof.